



北京大學

PEKING UNIVERSITY

SWIFTYPOPL: **A Swift-like Surface Language with Protocol** **Elaboration**

Runbang Yan

School of Computer Science, Peking University

TAPL Final Presentation

Table of Contents

① Motivation

② Language

③ Elaboration

④ Metatheory

⑤ Implementation

⑥ Conclusion



Why Protocol-Oriented Programming?

- Traditional OOP reuses code through class inheritance.
- This often creates fragile base classes, deep hierarchies, and early design commitments.
- Modern languages increasingly use interface-style abstraction: Go interfaces, Rust traits, and Swift protocols.
- Swift-style protocols combine several type-theoretic ideas: parametric polymorphism, ad-hoc polymorphism, first-class polymorphic functions, and existential values.



Project Goal

Goal

Design and implement a small Swift-like language whose protocol features can be elaborated into a standard typed core calculus.

- Surface language: structs, protocols, extensions, constrained generics, first-class functions, and any P.
- Core language: System F with records, existentials, conditionals, and natural-number primitives.
- Main metatheoretic claim:

well-typed surface program $\implies$ well-typed elaborated core program.



A Small Example

```
protocol Named {  
  func name() -> String  
}  
  
struct Screen {  
  let identifier: Int  
}  
  
extension Screen: Named {  
  func name() -> String { "Home" }  
}  
  
func keepNamed<T: Named>(item: T) -> T {  
  item  
}  
  
let screen: Screen = Screen(identifier: 1)  
let kept: Screen = keepNamed<Screen>(item: screen)  
kept.name()
```



Surface Fragment

- Nominal structs:

$$\text{struct } S \{ \overline{\text{let } x : \tau; \overline{md}} \}$$

- Protocol requirements:

$$\text{protocol } P \{ \overline{m : \sigma} \}$$

- Conformances via extensions:

$$\text{extension } S : P \{ \overline{md} \}$$

- Generic function schemes:

$$\overline{\forall X : \overline{P}. (\overline{\ell : \tau}) \rightarrow \tau}$$

- Existential protocol values:

$$\text{any } P$$



Typing Environments

$$\Psi = (\Psi_S, \Psi_P, \Psi_C, \Psi_M, \Psi_V)$$

Component	Meaning
$\Psi_S(S)$	fields of struct S
$\Psi_P(P)$	requirements of protocol P
$\Psi_C(S, P)$	conformance methods for $S : P$
$\Psi_M(S)$	methods available on struct S
$\Psi_V(x)$	top-level values and functions

- Ω tracks generic protocol constraints, such as $X : P$.
- Γ tracks local value variables.
- ρ resolves Self in struct, extension, and protocol contexts.

Protocol Satisfaction



Only two kinds of surface types satisfy protocols:

$$\frac{(S, P) \in \text{dom}(\Psi_C)}{\Psi; \Omega \vdash S \triangleright P} \text{SAT-STRUCT} \quad \frac{X: \bar{P} \in \Omega \quad P \in \bar{P}}{\Psi; \Omega \vdash X \triangleright P} \text{SAT-TVAR}$$

- A concrete struct satisfies a protocol through an extension.
- A type variable satisfies a protocol through a generic constraint.
- An existential value any P does *not* itself satisfy P .
- To use its dictionary, elaboration must open the existential package.

Core Target



Core calculus

System F + records + existentials + conditionals + recursion and natural-number primitives.

$$\begin{aligned}
 A &::= \text{Nat} \mid \text{Bool} \mid \text{String} \mid \text{Unit} \mid X \mid A \rightarrow A \mid \forall X.A \mid \{\overline{l : A}\} \mid \exists X.A \\
 t &::= x \mid \lambda x : A.t \mid t \, t \mid \Lambda X.t \mid t[A] \mid \{\overline{l = t}\} \mid t.l \mid \text{pack} \mid \text{unpack} \dots
 \end{aligned}$$

The surface checker handles Swift-like syntax; elaboration removes protocols by making evidence explicit.



Type Translation

$$\begin{aligned}
 \llbracket \text{Int} \rrbracket &= \text{Nat} \\
 \llbracket S \rrbracket &= \overline{\{x : \llbracket \tau \rrbracket\}} \quad \text{where } \Psi_S(S) = \{\overline{x : \tau}\} \\
 \llbracket \text{any } P \rrbracket &= \exists X. \{\text{value} : X, \text{dict} : \text{Dict}(P, X)\} \\
 \llbracket \overline{\forall X_i : \overline{P_i}. (\overline{\ell_j : \tau_j}) \rightarrow \tau_r} \rrbracket &= \forall \overline{X_i}. \overline{\llbracket X_i : \overline{P_i} \rrbracket \rightarrow \llbracket \tau_1 \rrbracket \rightarrow \cdots \rightarrow \llbracket \tau_r \rrbracket}
 \end{aligned}$$

- Each generic constraint $X : P$ becomes a dictionary parameter of type $\text{Dict}(P, X)$.
- A protocol dictionary is a record of methods:

$$\text{Dict}(P, A) = \overline{\{m_i : \llbracket \sigma_i \rrbracket^A\}}.$$



Elaboration as Dictionary Passing

Surface feature	Core representation
Protocol declaration	dictionary record type
Extension conformance	dictionary value
Constrained generic	explicit dictionary parameter
Method call on S	top-level method function + receiver
Method call on $X : P$	dictionary projection + receiver
any P value	existential package with value and dictionary

$$e \text{ as any } P \rightsquigarrow \text{pack}[[\tau], \{\text{value} = t, \text{dict} = d\}]$$



Existential Method Selection

$$\llbracket \text{any } P \rrbracket = \exists X. \{ \text{value} : X, \text{dict} : \text{Dict}(P, X) \}$$

For $e.m$, where $e : \text{any } P$:

$$M_{\exists} = \Lambda \bar{Y}. \overline{\lambda d_Y : \text{Dict}(Q, Y). \lambda y : \llbracket \tau \rrbracket.}$$

$$\text{unpack } \{X, z\} = t \text{ in } z.\text{dict}.m[\bar{Y}] \overline{d_Y} z.\text{value } \bar{y}$$

- Method-local type parameters are preserved.
- The hidden witness type X is used only inside the unpack body.
- Safety condition: visible argument and result types must not mention protocol `Self`.

What We Prove



Elaboration preservation

If a surface expression, declaration, or program type checks, then its elaborated core term has the translated type.

$$\begin{aligned} \Psi; \Omega; \Gamma; \rho \vdash e : \tau \quad \text{and} \quad \Psi; \Omega; \Gamma; \rho; \Delta \vdash e : \tau \rightsquigarrow t \\ \implies \mathcal{G}_\Psi; \llbracket \Omega \rrbracket; \llbracket \Gamma \rrbracket; \llbracket \Delta \rrbracket \vdash_c t : \llbracket \tau \rrbracket. \end{aligned}$$

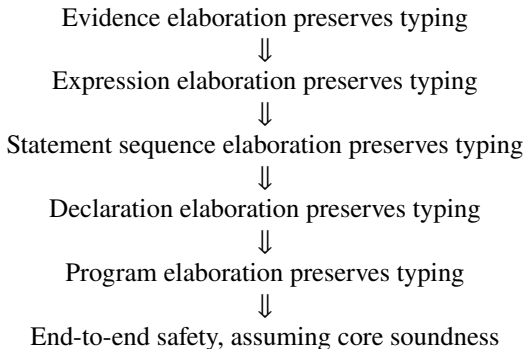
Core progress and preservation are assumed as standard TAPL-style results for the target calculus.



Key Lemmas

- 1 Surface type translation is defined.
- 2 Type translation preserves well-formedness.
- 3 Dictionary types are well formed.
- 4 Context translation is well formed.
- 5 Translation commutes with type substitution.
- 6 Resolved Self substitution matches self-parameterized translation.
- 7 Self-free types are stable under opening existentials.
- 8 Argument elaboration preserves typing.
- 9 Function wrappers preserve typing.

Main Theorem Chain



The proof is by induction over elaboration derivations, with standard structural properties such as weakening, exchange, and alpha-renaming.



Interesting Case: Generic Application

Surface type application checks constraints:

$$e : \overline{\forall X_i : \overline{P_i}. (\ell_j : \tau_j) \rightarrow \tau_r} \quad \Psi; \Omega \vdash \tau'_i \triangleright P$$

Elaboration inserts dictionary evidence:

$$e\langle \overline{\tau'_i} \rangle \quad \rightsquigarrow \quad t[\llbracket \overline{\tau'_i} \rrbracket] \overline{d_{i,P}}$$

- Evidence preservation gives $d_{i,P} : \text{Dict}(P, \llbracket \tau'_i \rrbracket)$.
- Core type application and function application yield the instantiated result.
- Translation-substitution commutation connects the core type back to the surface result.



Interesting Case: Existentials

For packaging:

$$\Psi; \Omega; \Gamma; \rho \vdash e : \tau \quad \Psi; \Omega \vdash \tau \triangleright P$$

$$e \text{ as any } P \quad \rightsquigarrow \quad \text{pack}[\llbracket \tau \rrbracket, \{\text{value} = t, \text{dict} = d\}]$$

For selection:

- Open the existential package.
- Project its dictionary.
- Call the method on the hidden receiver.
- Use the Self-free side condition to prevent the hidden type from escaping.



Implementation Pipeline

source file → lexer/parser → resolver → checker
→ elaborator → core checker/evaluator

- Resolver collects nominal names and resolves Self.
- Checker builds global environments, checks conformances, matches labels, and enforces existential safety.
- Elaborator performs dictionary-passing translation and renders core syntax.
- Core interpreter type checks and evaluates the elaborated program.



Supported Examples

- Table-view style delegates using existential protocol values.
- iOS settings screen model with existential delegates.
- First-class polymorphic identity functions.
- Struct fields containing polymorphic functions.
- Constrained generic functions elaborated with dictionary passing.
- Existential-safe method calls, including method-local type parameters.

Run

```
moon run src -- examples/<file>.swift
```



Takeaways

- SWIFTYPOPL isolates a proof-relevant subset of Swift-style protocol-oriented programming.
- Protocol constraints are compiled into explicit dictionary parameters.
- Existential protocol values are encoded using ordinary existential types.
- The paper proof shows that elaboration preserves typing.
- Therefore, surface safety follows from the assumed soundness of the core calculus.

Thank you!

Questions?